

Unit 04

Linux & the Command Line

Module 1 — Technical Foundations · ~5 class periods

Why this unit matters

- Almost every **security tool** you'll use (Kali, nmap, Burp) runs on Linux.
- Almost every **server** you'll ever test is Linux.
- PEN-200 (the pro pentest course) **assumes** you can already move around a shell.

If you can drive the command line, the rest of this course opens up.

GUI vs. command line

	Mouse (GUI)	Command line (CLI)
Speed	One click at a time	Many files at once
Repeat	Click again, by hand	Save it, run it again
Remote	Needs a desktop	Works over SSH

The CLI is how you control a machine you can't see.

The mindset for this unit

- The blank prompt is normal. **Type, read the output, adjust.**
- Nobody memorizes every command — pros look things up constantly.
- Small wins stack: each command you learn is a building block.

Treat errors as feedback, not failure. The shell is talking to you.

Learning objectives (1 of 2)

By the end of this unit you can:

- **Explain** why Linux and the CLI matter in security.
- **Navigate** the filesystem with `pwd`, `ls`, `cd`.
- **Create, view, copy, move, delete** files: `touch`, `cat`, `less`, `head`, `tail`, `cp`, `mv`, `rm`, `mkdir`.
- **Find** files and content: `find`, `locate`, `which`, `grep`.

Learning objectives (2 of 2)

- **Read and change** permissions (`rwX`) with `chmod` / `chown`; explain `root` and `sudo`.
- **Inspect** users and processes: `whoami`, `id`, `ps`, `top`, `kill`.
- **Install** software with `apt`; explain a package manager.
- **Combine** commands with pipes and redirection: `|`, `>`, `>>`, `<`.
- **Get help** for any command with `man` and `--help`.
- **Complete** OverTheWire Bandit levels 0-~12 and **document** your commands.

Key vocabulary (1 of 3)

Term	Meaning
Linux	Free, open-source OS running most servers + security tools.
Shell	Program that reads and runs the commands you type (e.g., Bash).
Terminal	The window/app where you type into the shell.
Command line (CLI)	Controlling the computer by typing, not clicking.
Filesystem hierarchy	The folder tree starting at <code>/</code> (root).
Path	The address of a file, e.g., <code>/home/student/notes.txt</code> .

Key vocabulary (2 of 3)

Term	Meaning
Absolute path	Starts from <code>/</code> — the full address.
Relative path	Starts from where you are now.
Working directory	The folder you are "in" right now (<code>pwd</code>).
Permissions (rwx)	Read / Write / eXecute, for owner / group / other.
<code>root</code>	The all-powerful administrator account.
<code>sudo</code>	"Superuser do" — run one command as administrator.
Process	A running program.

Key vocabulary (3 of 3)

Term	Meaning
Package manager	Tool that installs/updates software (e.g., <code>apt</code>).
Pipe <code> </code>	Sends one command's output into the next.
Redirection <code>></code> <code>>></code> <code><</code>	Send output to a file / append / take input from a file.
<code>man</code> / <code>--help</code>	The manual page / quick help for a command.
SSH	Secure Shell — encrypted remote login to another machine.



The one rule, again

Authorization and scope are the line.

The exact same commands are a **game** on an authorized target and a **crime** on a system you don't own.

Ethics for this unit

- Our playground is **OverTheWire Bandit** — published *specifically* for practice. That is your **written permission**, and your **scope** is the Bandit servers only.
- The same `ssh`, `find`, `grep` skills would be **illegal** against the school's servers, a classmate's account, or any machine outside this lab.
- Unauthorized access violates the **CFAA** and state law. **Minors are not exempt.**

Discussion: Hunting a password file on Bandit is the whole game and it's allowed. Same commands on the school file server — what changed? Why is one a game and one a crime?

Day 1

Why Linux? The shell, and getting around

What is a shell?

- The **shell** is the program that reads your typed commands and runs them. Bash is the most common one.
- The **terminal** is the window the shell lives in.
- You type a command, press Enter, **read the output**, try again. That loop is the whole skill.

Normalize the blank prompt: type, read, adjust. Nobody memorizes everything.

The filesystem tree

Everything starts at  — the **root** of the tree.

```
/          <- the root of everything
├─ home/   <- users' personal folders
│   └─ student/
├─ etc/    <- system configuration files
├─ bin/    <- programs (commands)
└─ var/    <- logs and changing data
```

Each folder can hold files and more folders. A **path** is an address into this tree.

Absolute vs. relative paths

Type	Example	Meaning
Absolute	<code>/home/student/notes.txt</code>	Full address from <code>/</code> . Works from anywhere.
Relative	<code>notes.txt</code> or <code>../etc</code>	Relative to where you are <i>now</i> .

- `.` means "here" · `..` means "one folder up" · `~` means "my home folder".

Knowing where you are: `pwd`

```
pwd
```

```
/home/student
```

- `pwd` = "print working directory" — shows the folder you're currently in.
- When you feel lost, `pwd` is your first move.

Listing what's there: `ls`

```
ls          # names only
ls -l       # long format: permissions, owner, size, date
ls -a       # show hidden files (names starting with .)
ls -la      # both: long format AND hidden files
```

- Hidden files start with a dot (`.bashrc`). They aren't secret — just tidy.

Moving around: `cd`

```
cd /etc          # go to an absolute path
cd Documents     # go into a subfolder (relative)
cd ..           # go up one level
cd ~             # go to your home folder
cd              # also goes home (no argument)
```

- After any `cd`, run `pwd` to confirm where you landed.

Worked walkthrough: getting around

```
pwd          # /home/student    (where am I?)
cd /etc      # jump to system config
pwd          # /etc              (confirm the move)
cd ..        # go up one level
pwd          # /                 (now at the root)
```

- Run `pwd` after every `cd` until it feels automatic.

Tab completion saves you

- Press **Tab** and the shell finishes a name for you.
- Type `cd /et` then **Tab** → it completes to `cd /etc/`.
- Press **Tab twice** to list all matching choices.

Tab completion means fewer typos and less typing. Use it constantly.

History and editing shortcuts

- **Up arrow** scrolls back through commands you already ran.
- **Ctrl+C** cancels a command that's running or stuck.
- **Ctrl+L** clears the screen (same as `clear`).
- `history` prints a numbered list of your recent commands.

Re-run a long command with Up arrow instead of retyping it.

Check your understanding (Day 1)

1. Which command prints the folder you're in right now?
2. What does `cd ..` do?
3. What does `~` stand for in a path?

Think it through before the next slide.

Answers (Day 1)

1. `pwd` — print working directory.
2. `cd ..` moves you **up one level** to the parent folder.
3. `~` is a shortcut for **your home folder** (e.g., `/home/student`).

Day 1 guided practice

Class "scavenger walk" — call out `pwd` at each stop:

```
cd /           # the very top
ls             # what lives at the root?
cd /home      # everyone's folders
cd /etc       # system config
cd ~          # back home
pwd           # confirm
```

Independent: explore your home folder with `pwd`, `ls -la`, and `cd`. Record three paths you visited.

Exit ticket: What command tells you where you are? What lists what's there?

Day 2

Working with files & finding things

Wildcards (globbing)

```
ls *.txt           # everything ending in .txt
ls report?.log     # report1.log, reportA.log (one char)
rm temp*          # everything starting with temp
```

- `*` matches any characters · `?` matches exactly one.
- The **shell** expands wildcards before the command even runs.

 Always `ls` a wildcard first to see what it matches before you `rm` it.

Creating files and folders

```
mkdir practice          # make a directory
touch notes.txt         # create an empty file (or update its time)
mkdir -p a/b/c          # make nested folders in one go
```

- `mkdir` = make directory · `touch` = create an empty file.

Copy, move, rename

```
cp notes.txt backup.txt      # copy
mv notes.txt archive.txt     # rename (move within same folder)
mv archive.txt practice/     # move into another folder
```

- `cp` copies · `mv` both **moves** and **renames** (same command).

Deleting — and why `rm` is scary

```
rm notes.txt          # delete a file
rm -r practice/       # delete a folder and everything in it
```



`rm` has **no recycle bin and no undo**. Once it's gone, it's gone.

- A mistyped `rm -rf` can erase huge amounts of data instantly.
- **Read before you delete.** Practice deletes only inside a `practice/` folder.

Viewing files

```
cat file.txt          # dump the whole file to the screen
less file.txt         # page through it (press q to quit)
head file.txt         # first 10 lines
tail file.txt         # last 10 lines
head -n 5 file.txt    # first 5 lines
```

- Use `cat` for short files, `less` for long ones so they don't scroll past.

Finding files: `find`, `locate`, `which`

```
find /home -name "*.txt"    # live search by name under /home
locate notes.txt            # fast search of a prebuilt index
which python3               # full path of a command's program
```

- `find` searches live (always current, can be slow).
- `locate` is fast but uses an index that may be stale.
- `which` tells you *where* a command lives.

Worked example: `find` by name

```
find /home -name "*.conf"
```

```
/home/student/.config/app.conf  
/home/student/old/backup.conf
```

- `find <where> -name "<pattern>"` walks the tree and prints matches.
- Quote the pattern so the **shell** doesn't expand `*` too early.

Searching inside files: `grep`

```
grep "password" file.txt      # lines containing "password"  
grep -r "flag" .              # search recursively in this folder  
grep -i "error" log.txt       # case-insensitive
```

- `find` finds **files**; `grep` finds **text inside** files.
- This pair is the heart of the Bandit wargame.

Worked example: `grep` output

```
grep "secret" notes.txt
```

```
the secret word is hydrogen  
my second secret is buried here
```

- `grep` prints **only the matching lines**, not the whole file.
- Add `-n` to also show the **line number** of each match.

Useful `grep` options

```
grep -n "error" log.txt      # show line numbers
grep -c "error" log.txt      # count matching lines
grep -v "debug" log.txt      # INVERT: lines WITHOUT "debug"
```

- `-i` ignore case · `-r` search folders · `-n` line numbers.
- `-v` is the odd one out: it shows everything that **doesn't** match.

Counting things: **wc**

```
grep "fail" log.txt | wc -l
```

```
12
```

- **wc -l** counts **lines**; **-w** words; **-c** characters.
- Piped after **grep**, it answers "how many matches?" in one line.

Check your understanding (Day 2)

1. Which tool finds **files by name**, and which finds **text inside** files?
2. Why is `rm` more dangerous than dragging a file to the trash?
3. What does `grep -r "flag" .` search?

Predict before flipping.

Answers (Day 2)

1. `find` finds files by name; `grep` finds text inside files.
2. `rm` deletes **immediately, with no recycle bin and no undo**.
3. The word `flag`, searched **recursively** in the current folder (`.`) and everything under it.

Day 2 guided practice

```
mkdir practice && cd practice
touch a.txt b.txt c.txt
echo "the secret word is hydrogen" > b.txt
cp b.txt copy.txt
mv copy.txt renamed.txt
grep -r "hydrogen" .          # which file has it?
```

Mini-challenge: hide a word in one of several files, then `grep -r` to find which file holds it.

Exit ticket: Difference between `cat` and `less`? Why is `rm` dangerous?

Day 3

Permissions, users & processes

Reading

`ls -l`

```
ls -l
-rwxr-xr--  1 student staff  512 Jun 4 notes.txt
```

The first 10 characters are the key:

```
-  rwx  r-x  r--
|  |    |    └─ others: read only
|  |    └────── group:  read + execute
|  └────────── owner:  read + write + execute
└────────── type: - = file, d = directory
```

The rwx model

Letter	On a file	Value
r	read the contents	4
w	change the contents	2
x	run it as a program	1

- There are **three groups**: **owner**, **group**, **others**.
- Each group gets its own rwx. `rwx` = $4+2+1 = 7$.

Permissions analogy: the building

- **Owner** = the person who rents the apartment.
- **Group** = the family members with a key.
- **Others** = everyone else in the world.

Each gets their own set of keys: read (look in), write (rearrange), execute (use the appliances).

Worked example: decode it

```
-rw-r--r--
```

Who	Bits	Can do
Owner	rw-	read + write
Group	r--	read only
Others	r--	read only

Nobody can execute it. This is a typical text file.

Worked example: a script

```
-rwxr-x---
```

Who	Bits	Can do
Owner	<pre>rwx</pre>	read + write + run
Group	<pre>r-x</pre>	read + run
Others	<pre>---</pre>	nothing

The leading

```
-
```

 means file; an

```
x
```

 means it can be **run** as a program.

Permissions as numbers

Each group's `rwX` adds up to a single digit:

Bits	Math	Number
<code>rwX</code>	$4+2+1$	7
<code>rw-</code>	$4+2$	6
<code>r-X</code>	$4+1$	5
<code>r--</code>	4	4

- So `chmod 750` = owner `rwX`, group `r-X`, others nothing.

Why permissions matter in security

- A world-writable script can be **edited by anyone** — a backdoor waiting to happen.
- A readable password file leaks secrets to every user on the box.
- Finding **wrong** permissions is a real way attackers escalate to root.

"Who can read, write, or run this?" is a question you'll ask all course.

Changing permissions: `chmod`

```
chmod +x script.sh      # add execute (make it runnable)
chmod 644 notes.txt      # owner rw-, group r--, others r--
chmod 755 program        # owner rwx, group r-x, others r-x
ls -l                   # confirm the change
```

- Two styles: **symbolic** (`+x`, `-w`) and **numeric** (`644`, `755`).
- `chown user file` changes who **owns** a file (usually needs `sudo`).

Worked example: `chmod` in action

```
ls -l script.sh
-rw-r--r-- 1 student student 64 Jun 4 script.sh
chmod +x script.sh
ls -l script.sh
-rwxr-xr-x 1 student student 64 Jun 4 script.sh
```

- Before: no `x`, can't run. After `chmod +x`: `x` appears for everyone.
- Confirm **every** permission change with a quick `ls -l`.

root and sudo

- **root** is the all-powerful admin account — it can change *anything*.
- A **normal user** is limited to their own files.
- **sudo** runs *one* command with admin power:

```
sudo apt update          # this needs admin rights
whoami                   # student
sudo whoami               # root
```

Least privilege: use only the access you need. Don't run everything as root — one mistake then breaks the whole system.

Users and processes

```
whoami      # which user am I?
id          # my user and group IDs
ps          # my running processes
ps aux      # every process on the system
top         # live, updating view (press q to quit)
```

- A **process** is just a running program, each with a **PID** (process ID).

Stopping a process: `kill`

```
ps aux | grep firefox    # find its PID
kill 4821                 # ask it to stop (by PID)
kill -9 4821              # force it to stop
```

- `kill` sends a signal to the process with that PID.
- `-9` is the forceful "stop no matter what" version.

Check your understanding (Day 3)

1. In `-rwxr-x---`, what can **others** do?
2. What number is `rw-r--r--` in `chmod` digits?
3. Why shouldn't you run every command with `sudo`?

Decode it on paper first.

Answers (Day 3)

1. **Nothing** — the last three bits are .
2. **644** (owner 6, group 4, others 4).
3. **Least privilege:** one mistake as root can break the whole system. Use admin power only when truly needed.

Day 3 practice

- Run `ls -l` and decode permissions for several files.
- `chmod` a file, then confirm with `ls -l`.
- Run `whoami`, `id`, `ps`; open `top`.
- **Decode three strings** like `-rwxr-xr--` into who can do what.

Exit ticket: What does `sudo` do, and why not run everything as root?

Day 4

Pipes, redirection, packages & getting help

Pipes: |

A **pipe** sends one command's output straight into the next command's input.

```
ls -l | grep txt      # list files, keep only lines with "txt"  
cat data.txt | head   # show the file, but only the top  
ps aux | grep ssh     # all processes, filtered to ssh
```

Small tools, chained together, become powerful. This is the Unix philosophy.

Building a pipeline step by step

```
ps aux                # 1. every process (huge list)
ps aux | grep ssh     # 2. keep only lines with "ssh"
ps aux | grep ssh | wc -l  # 3. count those lines
```

- Each `|` hands the output left → right.
- Build pipelines **one stage at a time**, checking output as you go.

Three streams: in, out, error

- **stdin** (0) — input · **stdout** (1) — normal output · **stderr** (2) — errors.
- `>` redirects stdout; `2>` redirects errors separately.
- `2>/dev/null` throws errors away while keeping real output.

You'll use `2>/dev/null` to silence "Permission denied" in Bandit.

Redirection:

```
ls -l > out.txt          # write output to a file (OVERWRITE)
ls -l >> out.txt         # append output to the file
grep "x" < input.txt     # feed a file in as input
```

Symbol	Action
	Send output to a file, replacing it
	Send output to a file, adding to the end
	Take input from a file

Worked example: redirect then read

```
ls /etc > files.txt      # write the listing to a file
wc -l files.txt          # how many lines did we save?
```

```
220 files.txt
```

- Nothing prints from the `ls` — it went **into** the file instead.
- `>` overwrites; run it again and the old contents are replaced.

Package managers and `apt`

- A **package manager** installs, updates, and removes software for you — and handles dependencies.
- On Debian / Ubuntu / Kali, that tool is `apt`.

```
sudo apt update           # refresh the list of available software
sudo apt install nmap     # install a program
sudo apt upgrade          # update installed software
```

- These need `sudo` (system-wide changes). Use carefully.

The most important skill: getting help

```
man find          # full manual (press q to quit)
find --help       # quick options summary
grep --help
```

You will never memorize every command and option. Knowing **how to look it up** is what makes you self-sufficient.

- Reach for `man` / `--help` *before* asking — and note what you learned.

Check your understanding (Day 4)

1. What does the pipe `|` do?
2. Difference between `>` and `>>`?
3. What does `2>/dev/null` hide, and why is that useful?

Answer before the reveal.

Answers (Day 4)

1. Sends one command's **output** into the next command's **input**.
2. `>` **overwrites** the file; `>>` **appends** to the end.
3. It discards **error messages** (stderr), so a noisy search shows only real results.

Day 4 practice + start Bandit

- Build a pipeline: `ls -la | grep <something>`.
- Redirect to a file: `... > out.txt`, then append with `>>`.
- Read a `man` page and find an option you didn't know.
- **Begin OverTheWire Bandit:** connect via SSH and do levels **0-3**.

Exit ticket: What does `|` do? What's the difference between `>` and `>>`?

Day 5

Bandit wargame — putting it together

Lab: OverTheWire Bandit

- A free, **pre-authorized** SSH wargame at overthewire.org.
- Each level's goal: **find the password for the next level**.
- Read the level page → figure out the command → run it → **log the command**.

The deliverable is the **command and your reasoning**, not the password.
Passwords rotate; the skill is forever.

Connecting with SSH

```
ssh bandit0@bandit.labs.overthewire.org -p 2220
```

- **Port is** `2220`, **not 22** — the #1 mistake people make.
- First connection: accept the host key by typing `yes`.
- Level 0 password is `bandit0`.
- You land at `bandit0@bandit:~$` — you're on the remote server.

Working a level (the loop)

1. Read the level page (it tells you which commands you'll need).
2. Figure out and run the command on the server.
3. Copy the password it reveals.
4. Log out and back in as the next user:

```
exit  
ssh bandit1@bandit.labs.overthewire.org -p 2220
```

What each level teaches (1 of 2)

Level	Skill
0 → 1	Read a normal file — <code>cat readme</code>
1 → 2	A file named <code>-</code> — use a path: <code>cat ./-</code>
2 → 3	Spaces in the name — quote: <code>cat "spaces in this filename"</code>
3 → 4	A hidden file — <code>ls -a</code> , then <code>cat</code> the dotfile
4 → 5	The one human-readable file — <code>file ./*</code> then <code>cat</code>
5 → 6	Find by size — <code>find inhere -size 1033c</code>

What each level teaches (2 of 2)

Level	Skill
6 → 7	Find by owner/group/size across <code>/</code> — <code>find / -user ... 2>/dev/null</code>
7 → 8	Word in a big file — <code>grep millionth data.txt</code>
8 → 9	The only unique line — <code>sort data.txt uniq -u</code>
9 → 10	Readable text in a binary — <code>strings data.txt grep =</code>
10 → 11	Base64 decode — <code>base64 -d data.txt</code>
11 → 12	ROT13 cipher — <code>cat data.txt tr 'A-Za-z' 'N-ZA-Mn-za-m'</code>

Worked level: a filename of `-`

```
ls          # shows a file literally named -  
cat -       # SEEMS to hang... why?  
cat ./-     # this works
```

- `cat -` reads from the **keyboard** (stdin), so it just waits.
- A path like `./-` makes clear you mean the **file** named `-`.

Worked level: which file is text?

```
file ./*          # ask what type each file is
./-file03: ASCII text
./-file07: data
cat ./-file03     # read the one that's text
```

- `file` reports the **type** of each file, not its name.
- Read only the human-readable one; ignore the binary "data" files.

Worked level: the unique line

```
sort data.txt | uniq -u
```

- `sort` groups identical lines next to each other.
- `uniq -u` prints only the lines that appear **exactly once**.
- Together they surface the one password hiding in noise.

Worked level: base64 decode

```
base64 -d data.txt
```

The password is Ab3...

- `base64` is **encoding**, not encryption — anyone can reverse it.
- `-d` means decode. The readable password falls right out.

Encoding hides nothing from someone who knows the format.

Check your understanding (Day 5)

1. What does the `file` command tell you?
2. Is base64 a form of encryption? Why does that matter?
3. Why log the **command**, not just the password?

Reason it out before the reveal.

Answers (Day 5)

1. The **type** of a file (ASCII text, data/binary, image, etc.).
2. **No** — it's reversible encoding. It hides nothing from anyone who knows the format.
3. Passwords **rotate**; the reusable skill is the command and your reasoning.

Hint: hide noise with `2>/dev/null`

```
find / -user bandit7 -group bandit6 -size 33c 2>/dev/null
```

- Searching all of `/` floods you with "Permission denied" messages.
- `2>/dev/null` throws away those **error** messages so you see only matches.
- This is redirection (`>`) applied to the **error** stream (`2`).

A good log entry

Level 3 → 4

Goal: read the hidden file in the inhere directory

Command: `ls -a inhere` then `cat inhere/.hidden`

Learned: hidden files start with a dot; `ls -a` shows them

- One entry per level: level, goal, **command used**, one line learned.
- Include at least one note where `man` / `--help` solved it for you.

When you get stuck

```
man find          # full manual
find --help       # quick options
grep --help
```

If a session hangs: `Ctrl+C` cancels, `Ctrl+D` or `exit` logs out. Each new level needs a fresh `ssh banditN@... -p 2220`.

Exit ticket: submit the Bandit log + one sentence: "the command I'm proudest of figuring out."

Lab deliverables

- Safety/authorization reminder restated in **your own words**.
- One entry **per level** (aim 0 → ~12): level, goal, command, one-line lesson.
- At least one note showing you used `man` or `--help`.
- A closing reflection: the level you're proudest of and why.

Stretch: push past level 12 · solve a level two ways · set up an SSH key · build a one-line pipeline.

Recap — what you can now do

- Navigate Linux: `pwd`, `ls`, `cd`, absolute vs. relative paths.
- Manage files: `touch`, `mkdir`, `cp`, `mv`, `rm`, `cat` / `less` / `head` / `tail`.
- Find things: `find`, `locate`, `which`, `grep`.
- Read/change permissions; understand `root`, `sudo`, least privilege.
- Inspect users/processes; install with `apt`; chain with `|` and `>`.
- Help yourself with `man` / `--help`.

Quiz preview (assessment)

- Which command shows your working directory? (`pwd`)
- What can the **group** do given `-rwxr-xr--`? (read + execute)
- Difference between `>` and `>>`? (overwrite vs. append)
- Why is `rm` dangerous vs. a GUI trash bin?
- Why does `cat -` seem to hang, and how do you read a file named `-`?

Discussion / exit ticket

On Bandit you log in and hunt for a password file — and it's allowed.

If you ran the exact same commands on the school's file server, what changes? Why is one a game and the other a crime?

Next up

Unit 05: Bash Scripting Basics — automate these commands and build a ping sweep.

Curriculum by Aj Hammond — PNPT, CRT0, OSCP, BSCP
github.com/ajm4n · linkedin.com/in/aj-hammond